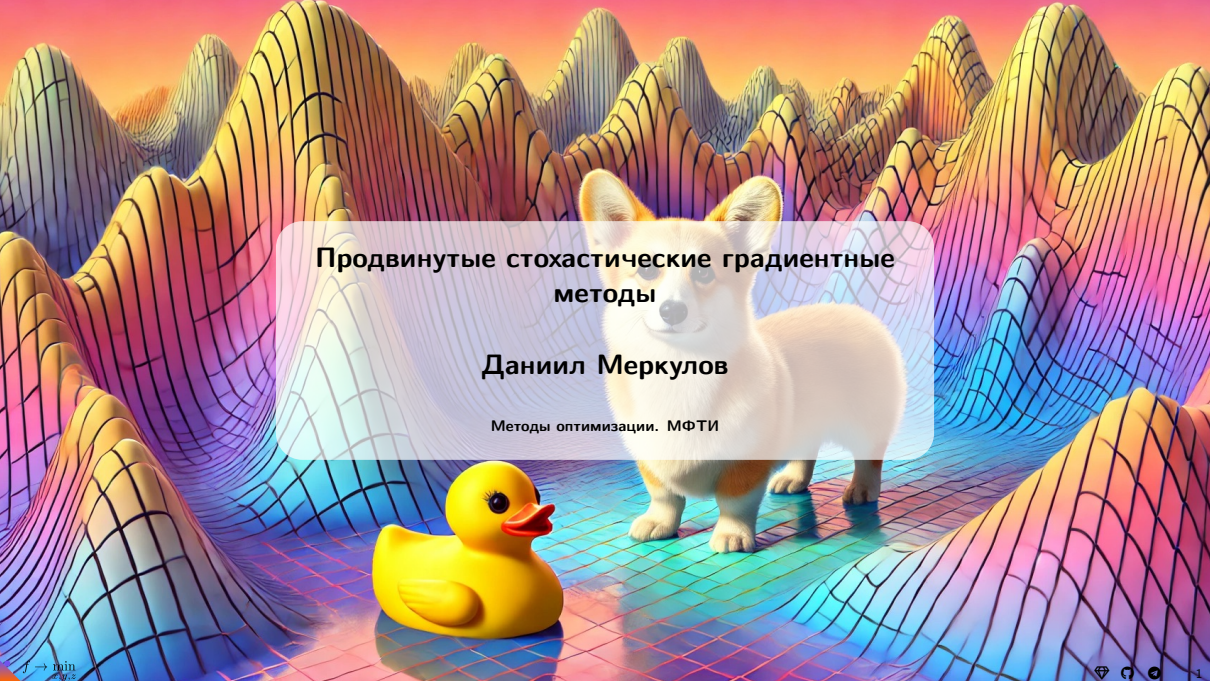


The background of the slide is a vibrant, stylized illustration. It features a series of colorful, wavy hills or mountains in shades of yellow, orange, pink, and blue. The surface of these hills is covered in a grid-like pattern of black lines. In the foreground, a small, fluffy Corgi dog with white and light brown fur stands on a similar grid-patterned surface. To the left of the dog, a bright yellow rubber duck is positioned. The overall scene is set against a soft, pastel-colored sky.

# Продвинутые стохастические градиентные методы

Даниил Меркулов

Методы оптимизации. МФТИ

## Вспоминаем задачу минимизации конечной суммы

## Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по  $n$ .

## Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по  $n$ .
- Сходимость при постоянном  $\alpha$  или линейном поиске.

## Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по  $n$ .
- Сходимость при постоянном  $\alpha$  или линейном поиске.

## Вспоминаем задачу минимизации конечной суммы

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск записывается следующим образом:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Стоимость итерации линейна по  $n$ .
- Сходимость при постоянном  $\alpha$  или линейном поиске.

Перейдём от полного вычисления градиента к его несмещённой оценке, случайно выбирая индекс  $i_k$  на каждой итерации равновероятно:

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k) \quad (\text{SGD})$$

При  $p(i_k = i) = \frac{1}{n}$  стохастический градиент является несмещённой оценкой градиента:

$$\mathbb{E}[\nabla f_{i_k}(x)] = \sum_{i=1}^n p(i_k = i) \nabla f_i(x) = \sum_{i=1}^n \frac{1}{n} \nabla f_i(x) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(x) = \nabla f(x)$$

Это означает, что математическое ожидание стохастического градиента равно истинному градиенту  $f(x)$ .

## Методы редукции дисперсии

## Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$



## Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2(\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения
  - Если  $\alpha < 1$ : потенциальное смещение (но уменьшенная дисперсия).

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения
  - Если  $\alpha < 1$ : потенциальное смещение (но уменьшенная дисперсия).
- Полезно, если  $Y$  положительно коррелирует с  $X$ .

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения
  - Если  $\alpha < 1$ : потенциальное смещение (но уменьшенная дисперсия).
- Полезно, если  $Y$  положительно коррелирует с  $X$ .

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2 (\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения
  - Если  $\alpha < 1$ : потенциальное смещение (но уменьшенная дисперсия).
- Полезно, если  $Y$  положительно коррелирует с  $X$ .

**Применение к оценке градиента:**

- SVRG: Пусть  $X = \nabla f_{i_k}(x^{(k-1)})$  и  $Y = \nabla f_{i_k}(\tilde{x})$ , при  $\alpha = 1$  и запомненном  $\tilde{x}$ .

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2(\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения
  - Если  $\alpha < 1$ : потенциальное смещение (но уменьшенная дисперсия).
- Полезно, если  $Y$  положительно коррелирует с  $X$ .

**Применение к оценке градиента:**

- SVRG: Пусть  $X = \nabla f_{i_k}(x^{(k-1)})$  и  $Y = \nabla f_{i_k}(\tilde{x})$ , при  $\alpha = 1$  и запомненном  $\tilde{x}$ .
- $\mathbb{E}[Y] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$  — полный градиент в  $\tilde{x}$ ;

# Ключевая идея редукции дисперсии

**Принцип:** уменьшение дисперсии выборки  $X$  с помощью выборки из другой случайной величины  $Y$  с известным мат. ожиданием:

$$Z_\alpha = \alpha(X - Y) + \mathbb{E}[Y]$$

- $\mathbb{E}[Z_\alpha] = \alpha\mathbb{E}[X] + (1 - \alpha)\mathbb{E}[Y]$
- $\text{var}(Z_\alpha) = \alpha^2(\text{var}(X) + \text{var}(Y) - 2\text{cov}(X, Y))$ 
  - Если  $\alpha = 1$ : нет смещения
  - Если  $\alpha < 1$ : потенциальное смещение (но уменьшенная дисперсия).
- Полезно, если  $Y$  положительно коррелирует с  $X$ .

**Применение к оценке градиента:**

- SVRG: Пусть  $X = \nabla f_{i_k}(x^{(k-1)})$  и  $Y = \nabla f_{i_k}(\tilde{x})$ , при  $\alpha = 1$  и запомненном  $\tilde{x}$ .
- $\mathbb{E}[Y] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$  — полный градиент в  $\tilde{x}$ ;
- $X - Y = \nabla f_{i_k}(x^{(k-1)}) - \nabla f_{i_k}(\tilde{x})$  — разность стремится к нулю вблизи решения.



# SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Хранить таблицу градиентов  $g_i$  от  $f_i$ ,  $i = 1, \dots, n$

## SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Хранить таблицу градиентов  $g_i$  от  $f_i$ ,  $i = 1, \dots, n$
- Инициализация  $x^{(0)}$ , и  $g_i^{(0)} = \nabla f_i(x^{(0)})$ ,  $i = 1, \dots, n$

## SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Хранить таблицу градиентов  $g_i$  от  $f_i$ ,  $i = 1, \dots, n$
- Инициализация  $x^{(0)}$ , и  $g_i^{(0)} = \nabla f_i(x^{(0)})$ ,  $i = 1, \dots, n$
- На шаге  $k = 1, 2, 3, \dots$ , выбрать случайный  $i_k \in \{1, \dots, n\}$ , затем

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{обновление последнего градиента } f_{i_k})$$

Все остальные  $g_i^{(k)} = g_i^{(k-1)}$ ,  $i \neq i_k$ , т.е. остаются без изменений

## SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Хранить таблицу градиентов  $g_i$  от  $f_i$ ,  $i = 1, \dots, n$
- Инициализация  $x^{(0)}$ , и  $g_i^{(0)} = \nabla f_i(x^{(0)})$ ,  $i = 1, \dots, n$
- На шаге  $k = 1, 2, 3, \dots$ , выбрать случайный  $i_k \in \{1, \dots, n\}$ , затем

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{обновление последнего градиента } f_{i_k})$$

Все остальные  $g_i^{(k)} = g_i^{(k-1)}$ ,  $i \neq i_k$ , т.е. остаются без изменений

- Обновление

$$x^{(k)} = x^{(k-1)} - \alpha_k \frac{1}{n} \sum_{i=1}^n g_i^{(k)}$$

## SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Хранить таблицу градиентов  $g_i$  от  $f_i$ ,  $i = 1, \dots, n$
- Инициализация  $x^{(0)}$ , и  $g_i^{(0)} = \nabla f_i(x^{(0)})$ ,  $i = 1, \dots, n$
- На шаге  $k = 1, 2, 3, \dots$ , выбрать случайный  $i_k \in \{1, \dots, n\}$ , затем

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{обновление последнего градиента } f_{i_k})$$

Все остальные  $g_i^{(k)} = g_i^{(k-1)}$ ,  $i \neq i_k$ , т.е. остаются без изменений

- Обновление

$$x^{(k)} = x^{(k-1)} - \alpha_k \frac{1}{n} \sum_{i=1}^n g_i^{(k)}$$

- Оценки градиента SAG **не являются несмещёнными**, но имеют значительно уменьшенную дисперсию

## SAG (Stochastic Average Gradient, Schmidt, Le Roux, Bach 2013)

- Хранить таблицу градиентов  $g_i$  от  $f_i$ ,  $i = 1, \dots, n$
- Инициализация  $x^{(0)}$ , и  $g_i^{(0)} = \nabla f_i(x^{(0)})$ ,  $i = 1, \dots, n$
- На шаге  $k = 1, 2, 3, \dots$ , выбрать случайный  $i_k \in \{1, \dots, n\}$ , затем

$$g_{i_k}^{(k)} = \nabla f_{i_k}(x^{(k-1)}) \quad (\text{обновление последнего градиента } f_{i_k})$$

Все остальные  $g_i^{(k)} = g_i^{(k-1)}$ ,  $i \neq i_k$ , т.е. остаются без изменений

- Обновление

$$x^{(k)} = x^{(k-1)} - \alpha_k \frac{1}{n} \sum_{i=1}^n g_i^{(k)}$$

- Оценки градиента SAG **не являются несмещёнными**, но имеют значительно уменьшенную дисперсию
- Не дорого ли усреднять все эти градиенты? Столь же эффективно, как SGD, при правильной реализации:

$$x^{(k)} = x^{(k-1)} - \alpha_k \underbrace{\left( \frac{1}{n} g_{i_k}^{(k)} - \frac{1}{n} g_{i_k}^{(k-1)} + \underbrace{\frac{1}{n} \sum_{i=1}^n g_i^{(k-1)}}_{\text{старое среднее таблицы}} \right)}_{\text{новое среднее таблицы}}$$

# Сходимость SAG

Предположим, что  $f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$ , где каждая  $f_i$  дифференцируема, и  $\nabla f_i$  липшицев с константой  $L$ .

Обозначим  $\bar{x}^{(k)} = \frac{1}{k} \sum_{l=0}^{k-1} x^{(l)}$  — среднюю итерацию после  $k - 1$  шагов.

## i Theorem

SAG с фиксированным шагом  $\alpha = \frac{1}{16L}$  и инициализацией

$$g_i^{(0)} = \nabla f_i(x^{(0)}) - \nabla f(x^{(0)}), \quad i = 1, \dots, n$$

удовлетворяет

$$\mathbb{E}[f(\bar{x}^{(k)})] - f^* \leq \frac{48n}{k} [f(x^{(0)}) - f^*] + \frac{128L}{k} \|x^{(0)} - x^*\|^2$$

# Сходимость SAG

Предположим, что  $f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$ , где каждая  $f_i$  дифференцируема, и  $\nabla f_i$  липшицев с константой  $L$ .

Обозначим  $\bar{x}^{(k)} = \frac{1}{k} \sum_{l=0}^{k-1} x^{(l)}$  — среднюю итерацию после  $k - 1$  шагов.

## i Theorem

SAG с фиксированным шагом  $\alpha = \frac{1}{16L}$  и инициализацией

$$g_i^{(0)} = \nabla f_i(x^{(0)}) - \nabla f(x^{(0)}), \quad i = 1, \dots, n$$

удовлетворяет

$$\mathbb{E}[f(\bar{x}^{(k)})] - f^* \leq \frac{48n}{k} [f(x^{(0)}) - f^*] + \frac{128L}{k} \|x^{(0)} - x^*\|^2$$

- Скорость  $\mathcal{O}\left(\frac{1}{k}\right)$  для SAG. Сравните:  $\mathcal{O}\left(\frac{1}{k}\right)$  для GD и  $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$  для SGD.



# Сходимость SAG

Предположим, что  $f(x) = \frac{1}{n} \sum_{i=1}^n f_i(x)$ , где каждая  $f_i$  дифференцируема, и  $\nabla f_i$  липшицев с константой  $L$ .

Обозначим  $\bar{x}^{(k)} = \frac{1}{k} \sum_{l=0}^{k-1} x^{(l)}$  — среднюю итерацию после  $k - 1$  шагов.

## i Theorem

SAG с фиксированным шагом  $\alpha = \frac{1}{16L}$  и инициализацией

$$g_i^{(0)} = \nabla f_i(x^{(0)}) - \nabla f(x^{(0)}), \quad i = 1, \dots, n$$

удовлетворяет

$$\mathbb{E}[f(\bar{x}^{(k)})] - f^* \leq \frac{48n}{k} [f(x^{(0)}) - f^*] + \frac{128L}{k} \|x^{(0)} - x^*\|^2$$

- Скорость  $\mathcal{O}\left(\frac{1}{k}\right)$  для SAG. Сравните:  $\mathcal{O}\left(\frac{1}{k}\right)$  для GD и  $\mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$  для SGD.
- Но константы различаются! Первый член в оценке SAG страдает от множителя  $n$ .

## Сходимость SAG. Сильно выпуклый случай

Предположим дополнительно, что каждая  $f_i$  сильно выпукла с параметром  $\mu$ .

### Theorem

SAG с шагом  $\alpha = \frac{1}{16L}$  удовлетворяет

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2}(f(x^{(0)}) - f^*) + \frac{4L}{n}\|x^{(0)} - x^*\|^2\right)$$

## Сходимость SAG. Сильно выпуклый случай

Предположим дополнительно, что каждая  $f_i$  сильно выпукла с параметром  $\mu$ .

### i Theorem

SAG с шагом  $\alpha = \frac{1}{16L}$  удовлетворяет

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2}(f(x^{(0)}) - f^*) + \frac{4L}{n}\|x^{(0)} - x^*\|^2\right)$$

- **Линейная** сходимость  $\mathcal{O}(\gamma^k)$  для SAG. Сравните:  $\mathcal{O}(\gamma^k)$  для GD и только  $\mathcal{O}\left(\frac{1}{k}\right)$  для SGD.

## Сходимость SAG. Сильно выпуклый случай

Предположим дополнительно, что каждая  $f_i$  сильно выпукла с параметром  $\mu$ .

### i Theorem

SAG с шагом  $\alpha = \frac{1}{16L}$  удовлетворяет

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2} (f(x^{(0)}) - f^*) + \frac{4L}{n} \|x^{(0)} - x^*\|^2\right)$$

- **Линейная** сходимость  $\mathcal{O}(\gamma^k)$  для SAG. Сравните:  $\mathcal{O}(\gamma^k)$  для GD и только  $\mathcal{O}\left(\frac{1}{k}\right)$  для SGD.
- SAG адаптивен к сильной выпуклости, как и GD.

## Сходимость SAG. Сильно выпуклый случай

Предположим дополнительно, что каждая  $f_i$  сильно выпукла с параметром  $\mu$ .

### i Theorem

SAG с шагом  $\alpha = \frac{1}{16L}$  удовлетворяет

$$\mathbb{E}[f(x^{(k)})] - f^* \leq \left(1 - \min\left(\frac{\mu}{16L}, \frac{1}{8n}\right)\right)^k \left(\frac{3}{2}(f(x^{(0)}) - f^*) + \frac{4L}{n}\|x^{(0)} - x^*\|^2\right)$$

- **Линейная** сходимость  $\mathcal{O}(\gamma^k)$  для SAG. Сравните:  $\mathcal{O}(\gamma^k)$  для GD и только  $\mathcal{O}(\frac{1}{k})$  для SGD.
- SAG адаптивен к сильной выпуклости, как и GD.
- Как и GD, SAG достигает линейной сходимости, но при стоимости итерации как у SGD.

## Замечания по SAG

- Метод в классической формулировке **не применим для больших нейросетей** из-за требований к памяти (нужно хранить  $n$  градиентов).

## Замечания по SAG

- Метод в классической формулировке **не применим для больших нейросетей** из-за требований к памяти (нужно хранить  $n$  градиентов).
- На практике можно использовать backtracking для оценки константы Липшица.

## Замечания по SAG

- Метод в классической формулировке **не применим для больших нейросетей** из-за требований к памяти (нужно хранить  $n$  градиентов).
- На практике можно использовать backtracking для оценки константы Липшица.
- Поскольку стохастический градиент  $g(x^k) \rightarrow \nabla f(x^k)$ , можно отслеживать сходимость по его норме (что неверно для SGD!).



## Замечания по SAG

- Метод в классической формулировке **не применим для больших нейросетей** из-за требований к памяти (нужно хранить  $n$  градиентов).
- На практике можно использовать backtracking для оценки константы Липшица.
- Поскольку стохастический градиент  $g(x^k) \rightarrow \nabla f(x^k)$ , можно отслеживать сходимость по его норме (что неверно для SGD!).
- Для обобщённых линейных моделей (LogReg, LLS) память  $\mathcal{O}(n)$  вместо  $\mathcal{O}(pn)$ :

$$f_i(w) = \varphi(w^T x_i) \leftrightarrow \nabla f_i(w) = \varphi'(w^T x_i) x_i$$

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$



# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
  - Обновить  $\tilde{x} = x_m$

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
  - Обновить  $\tilde{x} = x_m$

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
  - Обновить  $\tilde{x} = x_m$

## Замечания:

- Два вычисления градиента на каждом внутреннем шаге.

# SVRG (Stochastic Variance Reduced Gradient)

- Инициализация:  $\tilde{x} \in \mathbb{R}^d$
- Для  $i_{\text{epoch}} = 1$  до число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - Для  $t = 1$  до длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
  - Обновить  $\tilde{x} = x_m$

## Замечания:

- Два вычисления градиента на каждом внутреннем шаге.
- Два параметра: длина эпохи + шаг  $\alpha$ .

# SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:**  $\tilde{x} \in \mathbb{R}^d$
- **Для**  $i_{\text{epoch}} = 1$  **до** число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - **Для**  $t = 1$  **до** длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
  - Обновить  $\tilde{x} = x_m$

## Замечания:

- Два вычисления градиента на каждом внутреннем шаге.
- Два параметра: длина эпохи + шаг  $\alpha$ .
- Линейная сходимость, простое доказательство.

# SVRG (Stochastic Variance Reduced Gradient)

- **Инициализация:**  $\tilde{x} \in \mathbb{R}^d$
- **Для**  $i_{\text{epoch}} = 1$  **до** число эпох
  - Вычислить все градиенты  $\nabla f_i(\tilde{x})$ ; сохранить  $\nabla f(\tilde{x}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\tilde{x})$
  - Инициализировать  $x_0 = \tilde{x}$
  - **Для**  $t = 1$  **до** длина эпохи ( $m$ )
    - Выбрать  $i_t \in \{1, \dots, n\}$  равномерно
    - $x_t = x_{t-1} - \alpha [\nabla f_{i_t}(x_{t-1}) - \nabla f_{i_t}(\tilde{x}) + \nabla f(\tilde{x})]$
  - Обновить  $\tilde{x} = x_m$

## Замечания:

- Два вычисления градиента на каждом внутреннем шаге.
- Два параметра: длина эпохи + шаг  $\alpha$ .
- Линейная сходимость, простое доказательство.
- Не требует хранения  $n$  градиентов (в отличие от SAG).

# Численный эксперимент: SGD vs SVRG

Логистическая регрессия с  $\ell_2$ -регуляризацией.  $m = 200, n = 10, \mu = 0.1$

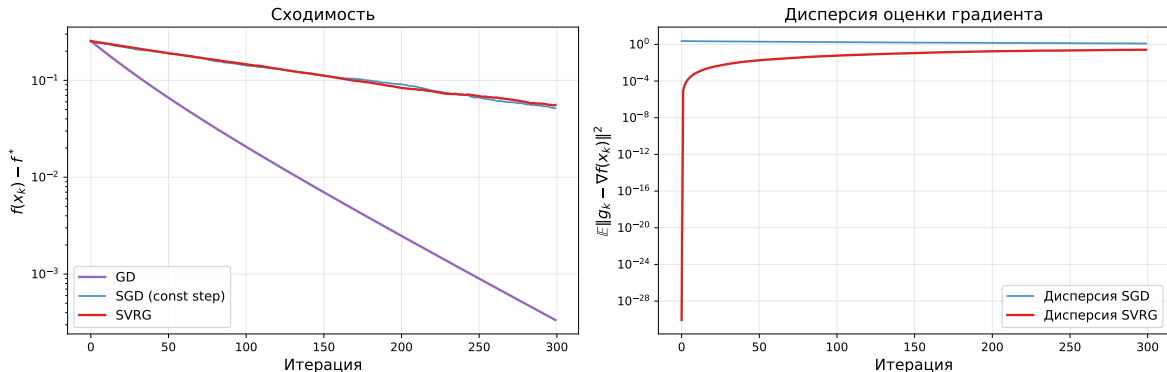


Рис. 1: Слева: сходимость  $f(x_k) - f^*$ . Справа: дисперсия оценки градиента. SVRG уменьшает дисперсию на порядки по мере приближения к решению, в то время как дисперсия SGD остаётся постоянной.

# Сравнение GD, SGD и SVRG

Strongly convex quadratics,  $m=150$ ,  $n=300$ ,  $\mu=1$ .

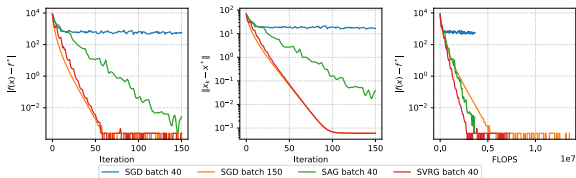


Рис. 2: Линейная регрессия с МНК

Strongly convex binary logistic regression,  $m=200$ ,  $n=10$ ,  $\mu=0.1$ .

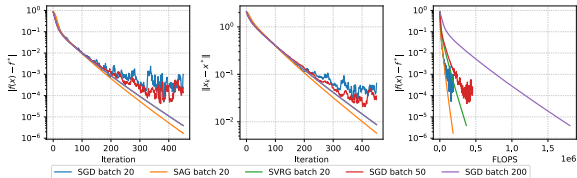


Рис. 3: Логистическая регрессия



# Сравнение GD, SGD и SVRG

Strongly convex quadratics,  $m=150$ ,  $n=300$ ,  $\mu=1$ .

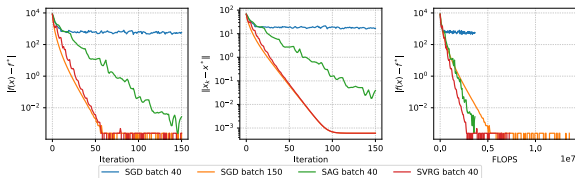


Рис. 2: Линейная регрессия с МНК

Strongly convex binary logistic regression,  $m=200$ ,  $n=10$ ,  $\mu=0.1$ .

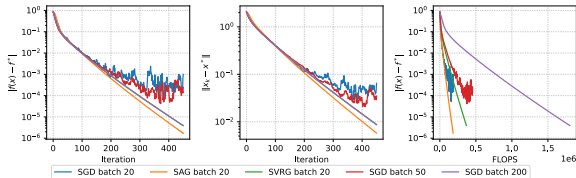


Рис. 3: Логистическая регрессия

| Метод | Стоимость итерации | Сходимость (сильно вып.)                               | Память  |
|-------|--------------------|--------------------------------------------------------|---------|
| GD    | $O(n)$             | линейная $(1 - \frac{\mu}{L})^k$                       | $O(p)$  |
| SGD   | $O(1)$             | сублинейная $\frac{1}{k}$                              | $O(p)$  |
| SAG   | $O(1)$             | линейная $(1 - \min(\frac{\mu}{16L}, \frac{1}{8n}))^k$ | $O(np)$ |
| SVRG  | $O(1)$             | линейная                                               | $O(p)$  |

## Адаптивные методы

## Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , а обновление для  $j = 1, \dots, p$  задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- AdaGrad не требует тонкой настройки шага:  $\alpha > 0$  задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.

## Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , а обновление для  $j = 1, \dots, p$  задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- AdaGrad не требует тонкой настройки шага:  $\alpha > 0$  задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.

## Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , а обновление для  $j = 1, \dots, p$  задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- AdaGrad не требует тонкой настройки шага:  $\alpha > 0$  задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.
- Может радикально превосходить SGD на разреженных задачах.

## Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , а обновление для  $j = 1, \dots, p$  задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- AdaGrad не требует тонкой настройки шага:  $\alpha > 0$  задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.
- Может радикально превосходить SGD на разреженных задачах.
- Главный недостаток — монотонное накопление градиентов в знаменателе. AdaDelta, Adam, AMSGrad и другие методы пытаются это исправить; они особенно популярны при обучении глубоких нейронных сетей.

## Adagrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , а обновление для  $j = 1, \dots, p$  задаётся так:

$$v_j^{(k)} = v_j^{k-1} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- AdaGrad не требует тонкой настройки шага:  $\alpha > 0$  задаётся как фиксированная константа, а эффективный шаг естественным образом убывает по мере итераций.
- Для редких, но информативных признаков шаг уменьшается медленно.
- Может радикально превосходить SGD на разреженных задачах.
- Главный недостаток — монотонное накопление градиентов в знаменателе. AdaDelta, Adam, AMSGrad и другие методы пытаются это исправить; они особенно популярны при обучении глубоких нейронных сетей.
- Константу  $\epsilon$  обычно берут равной  $10^{-6}$ , чтобы избежать деления на ноль и слишком больших шагов.

## RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, которое борется с его слишком агрессивным монотонным уменьшением шага. Использует скользящее среднее квадратов градиентов, чтобы адаптировать шаг для каждого параметра. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , тогда обновление для  $j = 1, \dots, p$  имеет вид:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- RMSPProp делит шаг для каждого параметра на скользящее среднее величин недавних градиентов этого параметра.



## RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, которое борется с его слишком агрессивным монотонным уменьшением шага. Использует скользящее среднее квадратов градиентов, чтобы адаптировать шаг для каждого параметра. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , тогда обновление для  $j = 1, \dots, p$  имеет вид:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- RMSPProp делит шаг для каждого параметра на скользящее среднее величин недавних градиентов этого параметра.
- Даёт более гибкую адаптацию шагов, чем AdaGrad, поэтому хорошо подходит для нестационарных задач.

## RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, которое борется с его слишком агрессивным монотонным уменьшением шага. Использует скользящее среднее квадратов градиентов, чтобы адаптировать шаг для каждого параметра. Пусть  $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$ , тогда обновление для  $j = 1, \dots, p$  имеет вид:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

### Замечания:

- RMSPProp делит шаг для каждого параметра на скользящее среднее величин недавних градиентов этого параметра.
- Даёт более гибкую адаптацию шагов, чем AdaGrad, поэтому хорошо подходит для нестационарных задач.
- Часто используется при обучении нейронных сетей, особенно рекуррентных.

## Adadelta (Zeiler, 2012)

Расширение RMSProp, которое пытается уменьшить зависимость от вручную заданного глобального шага. Вместо накопления всех прошлых квадратов градиентов Adadelta ограничивает окно накопления некоторым фиксированным размером  $w$ . Механизм обновления не требует явного задания шага  $\alpha$ :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

### Замечания:

- Adadelta адаптирует шаги, опираясь на скользящее окно обновлений градиента, а не на сумму всех прошлых градиентов. За счёт этого шаги устойчивее к изменениям динамики модели.

## Adadelta (Zeiler, 2012)

Расширение RMSProp, которое пытается уменьшить зависимость от вручную заданного глобального шага. Вместо накопления всех прошлых квадратов градиентов Adadelta ограничивает окно накопления некоторым фиксированным размером  $w$ . Механизм обновления не требует явного задания шага  $\alpha$ :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

### Замечания:

- Adadelta адаптирует шаги, опираясь на скользящее окно обновлений градиента, а не на сумму всех прошлых градиентов. За счёт этого шаги устойчивее к изменениям динамики модели.
- Метод не требует выбора начального шага, что упрощает настройку.

## Adadelta (Zeiler, 2012)

Расширение RMSProp, которое пытается уменьшить зависимость от вручную заданного глобального шага. Вместо накопления всех прошлых квадратов градиентов Adadelta ограничивает окно накопления некоторым фиксированным размером  $w$ . Механизм обновления не требует явного задания шага  $\alpha$ :

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$\tilde{g}_j^{(k)} = \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)}$$

$$x_j^{(k)} = x_j^{(k-1)} - \tilde{g}_j^{(k)}$$

$$\Delta x_j^{(k)} = \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2$$

### Замечания:

- Adadelta адаптирует шаги, опираясь на скользящее окно обновлений градиента, а не на сумму всех прошлых градиентов. За счёт этого шаги устойчивее к изменениям динамики модели.
- Метод не требует выбора начального шага, что упрощает настройку.
- Часто применяется в глубоком обучении, где масштабы параметров могут сильно различаться между слоями.

## Adam (Kingma and Ba, 2014) <sup>1 2</sup>

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

**Замечания:**

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.

# Adam (Kingma and Ba, 2014) <sup>1 2</sup>

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

**Замечания:**

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире

# Adam (Kingma and Ba, 2014) <sup>1 2</sup>

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

**Замечания:**

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе



## Adam (Kingma and Ba, 2014) <sup>1 2</sup>

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

**Замечания:**

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе
- Не сходится для некоторых простых задач (даже выпуклых)

## Adam (Kingma and Ba, 2014) <sup>1 2</sup>

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

**Замечания:**

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе
- Не сходится для некоторых простых задач (даже выпуклых)
- Почему-то очень хорошо работает для некоторых сложных задач

# Adam (Kingma and Ba, 2014)<sup>1 2</sup>

Сочетает идеи AdaGrad и RMSProp. Использует экспоненциально затухающие средние прошлых градиентов и квадратов градиентов.

EMA:

$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

Поправка на смещение:

$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Обновление:

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

**Замечания:**

- Исправляет смещение начальных моментов к нулю, которое есть у методов вроде RMSProp, и тем самым делает оценки точнее.
- Одна из самых цитируемых научных работ в мире
- В 2018-2019 годах вышли статьи, указывающие на ошибку в оригинальной работе
- Не сходится для некоторых простых задач (даже выпуклых)
- Почему-то очень хорошо работает для некоторых сложных задач
- Гораздо лучше работает для языковых моделей, чем для задач компьютерного зрения: почему?

---

<sup>1</sup>Adam: A Method for Stochastic Optimization

<sup>2</sup>On the Convergence of Adam and Beyond

## AdamW (Loshchilov & Hutter, 2017)

Решает типичную проблему  $\ell_2$ -регуляризации в адаптивных оптимизаторах вроде Adam. При стандартной  $\ell_2$ -регуляризации к функции потерь добавляется  $\lambda\|x\|^2$ , что даёт в градиенте дополнительный член  $\lambda x$ . В Adam этот член масштабируется адаптивным шагом  $\left(\sqrt{\hat{v}_j} + \epsilon\right)$ , из-за чего убывание весов оказывается связано с величинами градиентов.

AdamW отделяет убывание весов от шага адаптации по градиенту.

Правило обновления:

$$\begin{aligned}m_j^{(k)} &= \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)} \\v_j^{(k)} &= \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2 \\ \hat{m}_j &= \frac{m_j^{(k)}}{1 - \beta_1^k}, \quad \hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k} \\ x_j^{(k)} &= x_j^{(k-1)} - \alpha \left( \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)\end{aligned}$$

Замечания:

- Слагаемое убывания весов  $\lambda x_j^{(k-1)}$  добавляется *после* адаптивного градиентного шага.

## AdamW (Loshchilov & Hutter, 2017)

Решает типичную проблему  $\ell_2$ -регуляризации в адаптивных оптимизаторах вроде Adam. При стандартной  $\ell_2$ -регуляризации к функции потерь добавляется  $\lambda\|x\|^2$ , что даёт в градиенте дополнительный член  $\lambda x$ . В Adam этот член масштабируется адаптивным шагом  $\left(\sqrt{\hat{v}_j} + \epsilon\right)$ , из-за чего убывание весов оказывается связано с величинами градиентов.

AdamW отделяет убывание весов от шага адаптации по градиенту.

Правило обновления:

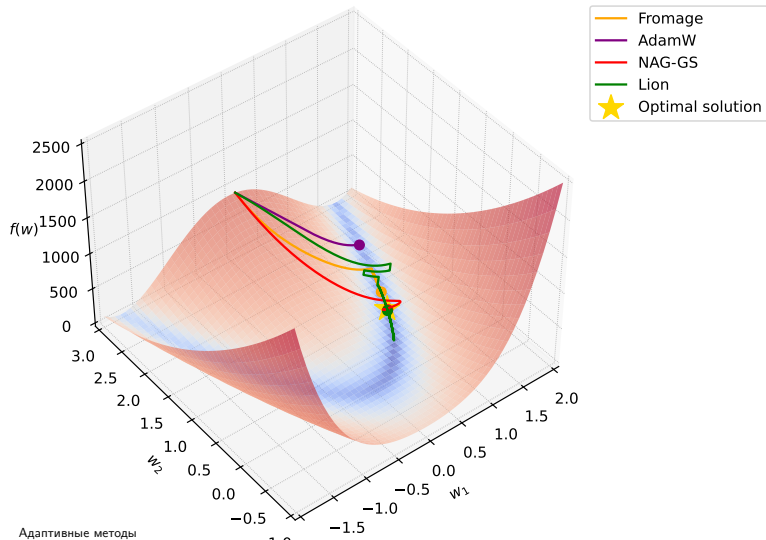
$$\begin{aligned}m_j^{(k)} &= \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)} \\v_j^{(k)} &= \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2 \\ \hat{m}_j &= \frac{m_j^{(k)}}{1 - \beta_1^k}, \quad \hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k} \\ x_j^{(k)} &= x_j^{(k-1)} - \alpha \left( \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)\end{aligned}$$

Замечания:

- Слагаемое убывания весов  $\lambda x_j^{(k-1)}$  добавляется *после* адаптивного градиентного шага.
- Широко используется при обучении трансформеров и других больших моделей. Часто является выбором по умолчанию в Hugging Face Trainer.

# Таких методов немало

Rosenbrock Function.  
Adaptive stochastic gradient algorithms.  
Learning rate 0.003



## Как их сравнивать? Бенчмарк AlgoPerf<sup>3 4</sup>

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:

## Как их сравнивать? Бенчмарк AlgoPerf <sup>3</sup> <sup>4</sup>

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
  - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.



# Как их сравнивать? Бенчмарк AlgoPerf <sup>3</sup> <sup>4</sup>

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
  - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.
  - **Self-Tuning:** моделирует автоматическую настройку на одной машине (фиксированная или inner-loop настройка, бюджет в 3 раза больше). Оценка строится по медиане времени работы на 5 исследованиях.

## Как их сравнивать? Бенчмарк AlgoPerf<sup>3 4</sup>

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
  - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.
  - **Self-Tuning:** моделирует автоматическую настройку на одной машине (фиксированная или inner-loop настройка, бюджет в 3 раза больше). Оценка строится по медиане времени работы на 5 исследованиях.
- **Система оценивания:** результаты по задачам агрегируются с помощью профилей производительности. Профиль показывает долю задач, решённых за время не более чем в  $\tau$  раз больше времени лучшей посылки. Итоговый балл — нормированная площадь под профилем (1.0 означает лучший результат на всех задачах).

## Как их сравнивать? Бенчмарк AlgoPerf<sup>3 4</sup>

- **Бенчмарк AlgoPerf:** сравнивает алгоритмы обучения нейронных сетей в двух режимах:
  - **External Tuning:** моделирует подбор гиперпараметров при ограниченных ресурсах (5 запусков, квазислучайный поиск). Оценка строится по медиане минимального времени достижения цели на 5 независимых исследованиях.
  - **Self-Tuning:** моделирует автоматическую настройку на одной машине (фиксированная или inner-loop настройка, бюджет в 3 раза больше). Оценка строится по медиане времени работы на 5 исследованиях.
- **Система оценивания:** результаты по задачам агрегируются с помощью профилей производительности. Профиль показывает долю задач, решённых за время не более чем в  $\tau$  раз больше времени лучшей посылки. Итоговый балл — нормированная площадь под профилем (1.0 означает лучший результат на всех задачах).
- **Вычислительная стоимость:** для оценивания потребовалось  $\sim 49,240$  суммарных часов на конфигурации из 8 NVIDIA V100 (в среднем  $\sim 3469$  ч на одну посылку в режиме External Tuning и  $\sim 1847$  ч на одну посылку в режиме Self-Tuning).

---

<sup>3</sup>Benchmarking Neural Network Training Algorithms

<sup>4</sup>Accelerating neural network training: An analysis of the AlgoPerf competition

# Бенчмарк AlgoPerf

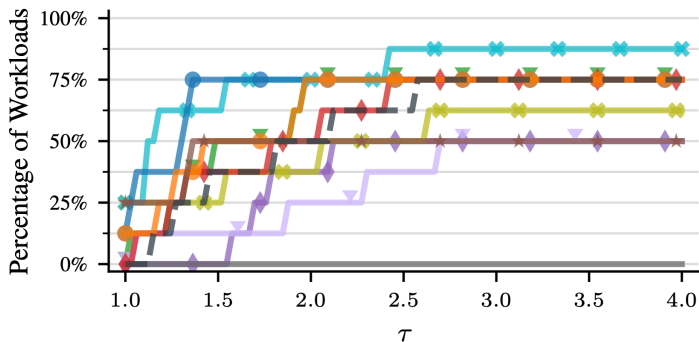
**Сводка по фиксированным базовым задачам в бенчмарке AlgoPerf.** В качестве функций потерь используются кросс-энтропия (CE), средняя абсолютная ошибка (L1) и потери CTC (Connectionist Temporal Classification). Дополнительные метрики качества: индекс структурного сходства (SSIM), частота ошибок (ER и WER), средняя точность (mAP) и BLEU. Бюджет времени соответствует режиму External Tuning; в режиме Self-Tuning обучение может идти в 3× дольше.

| Задача                       | Датасет     | Модель      | Потери | Метрика | Цель     | Бюджет |
|------------------------------|-------------|-------------|--------|---------|----------|--------|
| Прогнозирование CTR          | CRITEO 1TB  | DLRMSMALL   | CE     | CE      | 0.123735 | 7703   |
| Реконструкция MPT            | FASTMRI     | U-NET       | L1     | SSIM    | 0.7344   | 8859   |
| Классификация изображений    | IMAGENET    | ResNet-50   | CE     | ER      | 0.22569  | 63,008 |
|                              |             | ViT         | CE     | ER      | 0.22691  | 77,520 |
| Распознавание речи           | LIBRISPEECH | Conformer   | CTC    | WER     | 0.085884 | 61,068 |
|                              |             | DeepSpeech  | CTC    | WER     | 0.119936 | 55,506 |
| Предсказание свойств молекул | OGBG        | GNN         | CE     | mAP     | 0.28098  | 18,477 |
| Машинный перевод             | WMT         | Transformer | CE     | BLEU    | 30.8491  | 48,151 |

# Бенчмарк AlgoPerf

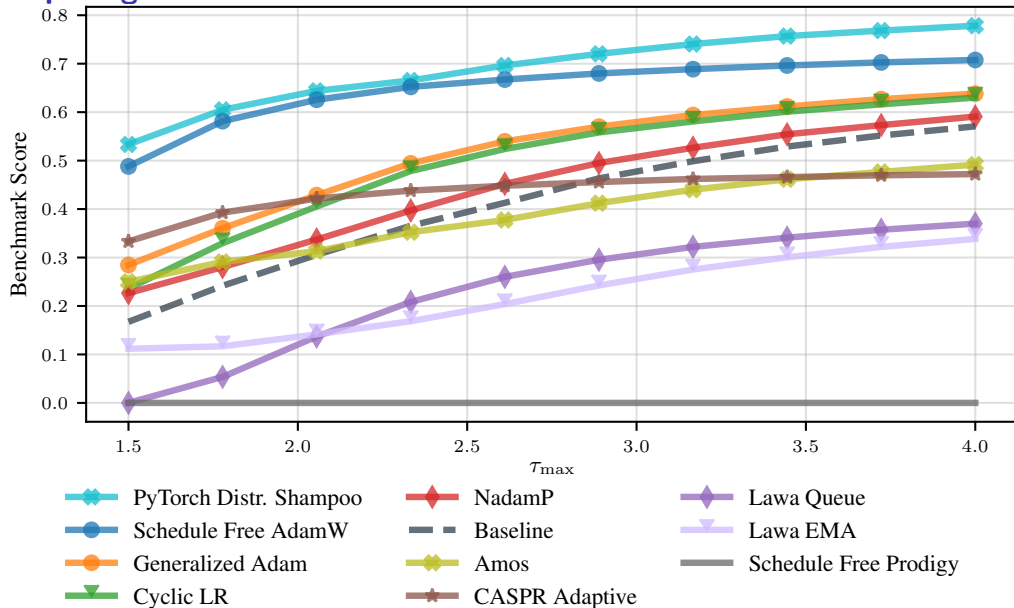
| Submission                  | Line                                                                              | Score  |
|-----------------------------|-----------------------------------------------------------------------------------|--------|
| PYTORCH DISTRIBUTED SHAMPOO |  | 0.7784 |
| SCHEDULE FREE ADAMW         |  | 0.7077 |
| GENERALIZED ADAM            |  | 0.6383 |
| CYCLIC LR                   |  | 0.6301 |
| NADAMP                      |  | 0.5909 |
| BASELINE                    |  | 0.5707 |
| AMOS                        |  | 0.4918 |
| CASPR ADAPTIVE              |  | 0.4722 |
| LAWA QUEUE                  |  | 0.3699 |
| LAWA EMA                    |  | 0.3384 |
| SCHEDULE FREE PRODIGY       |  | 0      |

(a) External tuning leaderboard



(b) External tuning performance profiles

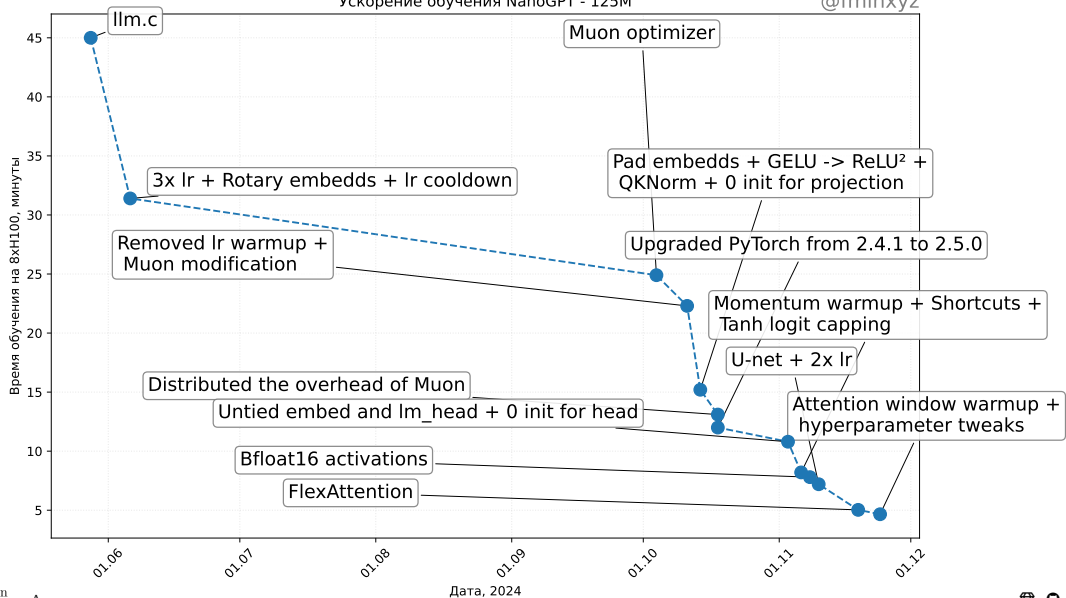
# Бенчмарк AlgoPerf



# Спидран NanoGPT

Ускорение обучения NanoGPT - 125M

@fminxyz



## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .

**Замечания:**



## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .

**Замечания:**

## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .
3. Вычислить предобуславливатели  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$ . (Обратный корень матрицы)

**Замечания:**

## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .
3. Вычислить предобуславливатели  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$ . (Обратный корень матрицы)
4. Обновить параметры:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .
3. Вычислить предобуславливатели  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$ . (Обратный корень матрицы)
4. Обновить параметры:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.

## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .
3. Вычислить предобуславливатели  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$ . (Обратный корень матрицы)
4. Обновить параметры:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.
- Вычислительно дороже Adam, но по числу шагов может сходиться быстрее или приходить к лучшим решениям.

## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .
3. Вычислить предобуславливатели  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$ . (Обратный корень матрицы)
4. Обновить параметры:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.
- Вычислительно дороже Adam, но по числу шагов может сходиться быстрее или приходить к лучшим решениям.
- Требуется аккуратной и эффективной реализации (например, быстрого вычисления обратных корней матриц и работы с большими матрицами).

## Shampoo (Gupta, Anil, et al., 2018; Anil et al., 2020)

Название расшифровывается как **S**tochastic **H**essian-**A**pproximation **M**atrix **P**reconditioning for **O**ptimization **O**f deep networks. Это метод, вдохновлённый оптимизацией второго порядка и ориентированный на крупномасштабное глубокое обучение.

**Основная идея:** аппроксимировать полное матричное предобуславливание AdaGrad с помощью эффективных матричных структур, в частности кронекеровских произведений.

Для весовой матрицы  $W \in \mathbb{R}^{m \times n}$  обновление использует предобуславливание через приближения статистических матриц  $L \approx \sum_k G_k G_k^T$  и  $R \approx \sum_k G_k^T G_k$ , где  $G_k$  — градиенты.

Упрощённая схема:

1. Вычислить градиент  $G_k$ .
2. Обновить статистики  $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$  и  $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$ .
3. Вычислить предобуславливатели  $P_L = L_k^{-1/4}$  и  $P_R = R_k^{-1/4}$ . (Обратный корень матрицы)
4. Обновить параметры:  $W_{k+1} = W_k - \alpha P_L G_k P_R$ .

**Замечания:**

- Стремится учитывать информацию о кривизне эффективнее, чем методы первого порядка.
- Вычислительно дороже Adam, но по числу шагов может сходиться быстрее или приходить к лучшим решениям.
- Требуется аккуратной и эффективной реализации (например, быстрого вычисления обратных корней матриц и работы с большими матрицами).
- Существуют варианты для тензоров разных форм, например для сверточных слоёв.

$$\begin{aligned}W_{t+1} &= W_t - \eta(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4} \\&= W_t - \eta(U S^2 U^\top)^{-1/4} (U S V^\top) (V S^2 V^\top)^{-1/4} \\&= W_t - \eta(U S^{-1/2} U^\top) (U S V^\top) (V S^{-1/2} V^\top) \\&= W_t - \eta U S^{-1/2} S S^{-1/2} V^\top \\&= W_t - \eta U V^\top\end{aligned}$$